

ICML 2023 | 把张量重计算问题「变小」，Moccasin 让大计算图求解快一个数量级

Qualcomm AI Research 与南加州大学提出基于约束规划的张量重计算新形式，把离散决策变量从平方级压到线性级

在手机、可穿戴设备、端侧加速器这类边缘硬件上跑神经网络，最先撞上的瓶颈往往不是算力，而是内存。片上内存容量有限，一旦放不下所有中间张量，就只能频繁访问速度慢得多的外部内存，直接拖慢推理和训练。

一个经典的应对办法是「张量重计算」（tensor rematerialization）：不把每个中间张量都存下来，而是在需要时重新算一遍，用额外的计算换取更低的峰值内存。问题在于，重算是要花时间的。于是就有了一个调度难题：给定一张计算图和固定的内存预算，到底该保留哪些张量、重算哪些张量，才能让总执行时间尽可能短，同时峰值内存始终不超预算？这在一般情况下是 PSPACE-complete 的组合优化问题。

此前的代表性方法 Checkmate 把它建模成一个混合整数线性规划（MILP），表达能力很强，但布尔决策变量的数量随节点数 n 呈平方级 $O(n^2)$ 增长。图一大，求解器要么超时、要么内存耗尽。本文提出的 Moccasin 换了一种建模思路：用「保留区间」（retention intervals）描述每个节点的张量生命周期，把离散变量的数量从平方级压到线性级 $O(n)$ ，从而在几百节点、上千条边的大图上，比 Checkmate 快最多约一个数量级。

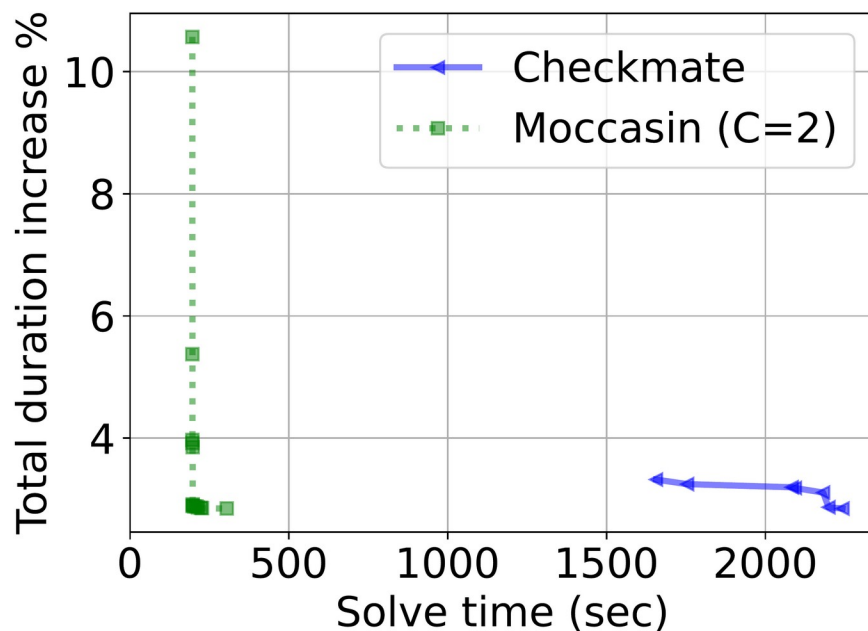


图 1 在一张 $n=442$ 个节点、 $m=1247$ 条边的真实计算图上，两种方法的「总时长增幅 vs. 求解时间」对比（内存预算设为无重计算时峰值内存的 80%）。Moccasin（绿色）在两三百秒内就收敛到低时长增幅的解，Checkmate（蓝色）要到约 2000 秒才给出同等质量的结果。

为什么重计算问题在大图上这么难求解

计算图里每个节点对应一次算子计算，产生一个中间张量。保留张量能省下重算的时间，却要一直占着内存；重算能省内存，代价是多花时间。内存预算正是把这个取舍变成一个约束优化问题的关键。Checkmate 的做法，是给「某个张量在某个时间步是否驻留内存」这件事都配一个布尔变量，于是变量数量随节点数和时间步数一起膨胀，达到 $O(n^2)$ 量级。这套表达在中小规模的图上很有效，但一旦图长到几百个节点，变量数就会爆炸，求解随之失控。

方法核心：用「保留区间」描述张量生命周期

Moccasin 的关键观察是：在整个执行过程里，一个张量其实只会经历少数几段「待在内存里」的时间。与其为每个时间步单独记一个布尔量，不如直接用区间来刻画这些驻留段。

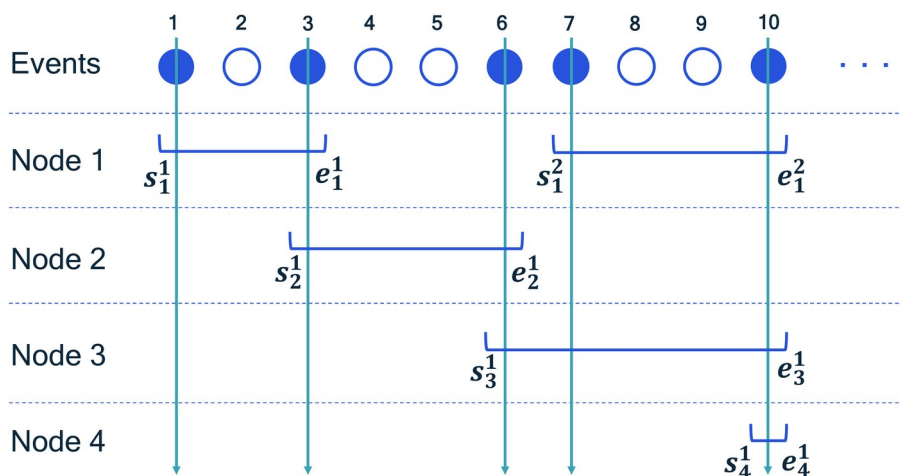


图 2 保留区间形式化的示意。横轴是被划分成「事件」（event）的时间，每一行对应一个节点。每段区间由一个起始事件 s 和一个结束事件 e 界定：区间的第一个事件表示重新计算该节点、把张量写入内存，区间结束表示张量被释放。图中节点 1 用了两段区间（说明它被重算了一次），其余节点各用一段。

每个节点最多允许 C_v 段区间，每段由整数起点、整数终点，以及一个布尔活跃标志描述。目标函数是最小化所有活跃计算事件的加权和，权重就是各节点的执行时间，也就是重算带来的总时长开销。内存预算用约束规划里的 cumulative 约束表达：把张量大小当作资源占用、把预算 M 当作容量；节点间的数据依赖则用 reservoir 约束刻画。整套模型交给 Google OR-Tools 的 CP-SAT 求解器求解。实验里把每个节点的区间上限 C_v 固定为 2，于是整数变量总数只有 $O(Cn)$ ，随图规模线性增长。

用「阶段」进一步收缩搜索空间

Moccasin 还提供一个可选变体：强制区间的起点对齐到按输入拓扑序划分出的「阶段」（stage）。

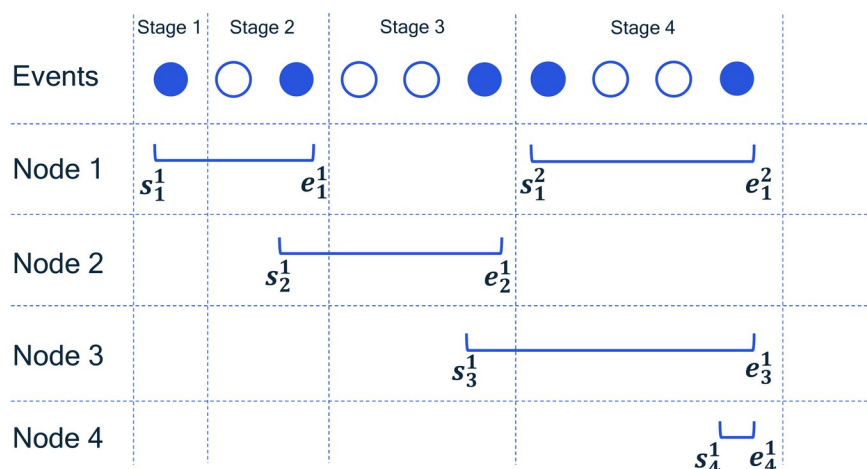


图 4 把事件按阶段分组的示意。当强制采用一个输入拓扑序时，每个节点第一段区间的起点被固定为定值，不再是自由变量，从而缩小整体搜索空间、加快求解。

论文的消息实验显示，这个拓扑序约束虽然会略微放大变量的取值域，却能显著压缩搜索空间，让求解更快；而把 C_v 压到 2 这么小，也几乎不损失解的质量。

实验结果：解的质量持平，求解时间大幅缩短

作者在一组随机分层图（random layered graphs）G1 到 G4（节点数从 100 到 1000，边数最多 5875）以及真实计算图上做了对比，内存预算取无重计算时峰值内存的 80% 和 90%。

表 1 随机分层图上的求解时间对比（内存预算为初始峰值内存的 80%）。「总时长增幅」指重计算方案相对无重计算方案增加的执行时间百分比；「—」表示方法在时限内未找到任何可行解。

计算图 (n, m)	Checkmate 求解时间 (s)	Moccasin 求解时间 (s)	总时长增幅 (Moccasin)
G1 (100, 236)	18.5	9.3	0.8%
G2 (250, 944)	685.1	55.0	0.9%
G3 (500, 2461)	—	1803.3	0.7%
G4 (1000, 5875)	—	3612.9	0.7%

可以看到两点。第一，在两种方法都能求解的图上（G1、G2），Moccasin 得到的解质量（总时长增幅）与 Checkmate 基本持平，但求解时间快了数倍到十几倍。第二，也是更关键的一点：当图长到 500、1000 个节点时，Checkmate 在三小时时限内找不到任何可行解、并因内存耗尽退出，而 Moccasin 仍能在一小时内收敛到总时长增幅低于 1% 的好解。值得一提的是，Moccasin 在整张表里给出的解，总时长增幅都稳定低于 1%，意味着为省内存而多付出的重算时间几乎可以忽略。图 1 那张真实图上的曲线，正是这种数量级差距的直观写照。

小结与边界

Moccasin 的启示很朴素：换一个更省变量的问题建模，往往比换一个更强的求解器更能改变「什么规模可解」。把重计算决策表达成少量保留区间，让离散变量从 $O(n^2)$ 降到

$O(n)$ ，Moccasin 就能处理此前方法根本啃不动的大计算图，求解速度最高快约一个数量级，同时把重计算带来的额外运行时开销始终压在 5% 以内，几乎不影响整体的执行效率。

也需要说明边界：这套方法优化的是「求解出执行序列」这一步，本身与硬件无关，解出来的序列可以在任意 CPU 或 GPU 上执行；但它目前只考虑重计算，尚未纳入「换页」（paging，把中间数据临时写到外部内存）这类策略，作者把与换页结合列为后续工作。对需要在编译期为大模型做内存规划的场景，Moccasin 提供了一个更能向大图扩展的选项。

落到工程实践上，收益最大的地方，恰恰是老方法失效的地方。为大模型做编译期内存规划，或者把训练与推理塞进显存、片上内存受限的设备，正是「大计算图」这一档：图越大，Moccasin 相对以往方法的优势越明显，而在小图上两者差距并不大。所以这套重新建模真正换来的，不是对每个案例都提速，而是把「根本能不能解」的边界往外推了一大截。